Will Your Model Fit?

Lesson 1 of 12 · GPUs in plain English: memory size, memory speed, compute and connection.

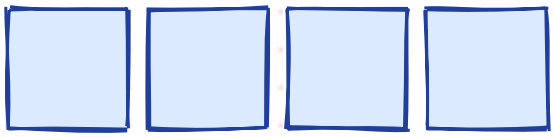
In this lesson → ① The four numbers ② Weights in gigabytes ③ Precision ④ The KV cache ⑤ Memory speed, measured ⑥ Batching ⑦ Interview Q&A

Course project: ShopNest Copilot, the AI support assistant we move to the cloud, step by step.

1 The cloud, for AI

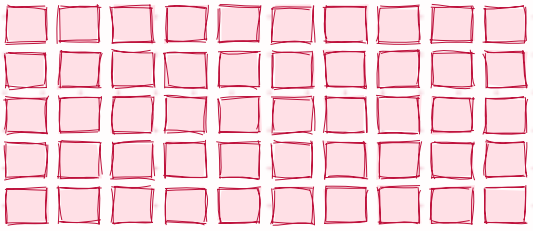
The cloud = **renting computers by the hour**. For AI the computers that matter are GPUs: too costly to buy, hard to get, so you rent the right ones for the time you need them.

CPU: a few strong workers



great at many different jobs

GPU: thousands of simple workers



all doing the same sum at once

AI is mostly one job: multiply big tables of numbers

→ A **CPU** has a few strong workers, good at many different jobs
→ A **GPU** has thousands of simple workers doing the same sum at once
→ AI is mostly one job: multiplying huge tables of numbers, so it fits a GPU perfectly

The rule of this course
A number I **measured** is marked measured. A number from a vendor is marked as theirs and dated (here: 14 Oct 2026). Arithmetic with an assumed model shape is labelled too.

CLOUD NOTEBOOK

2 The four numbers

1
memory size
what fits

2
memory speed
how fast chat runs

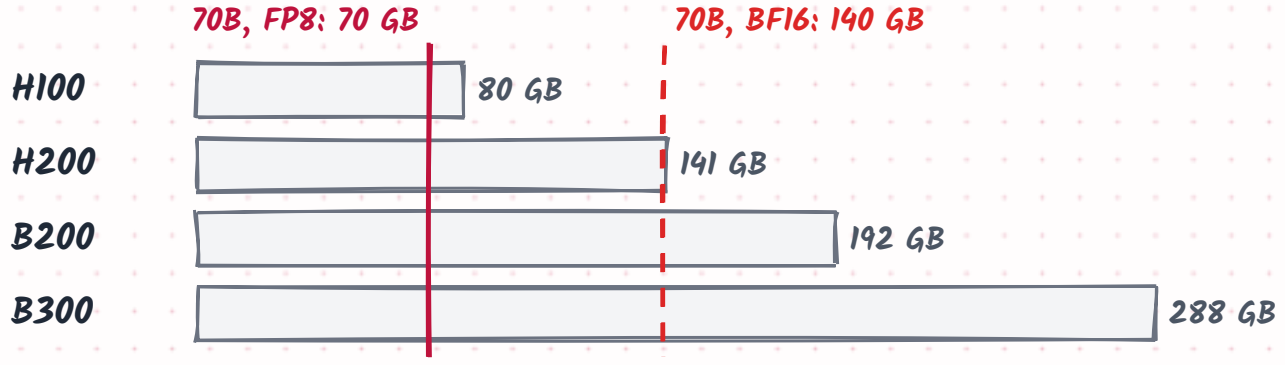
3
compute
how fast it thinks

4
connection
how GPUs talk

Number	What it tells you	Lesson
1 · memory size	what fits on one GPU	this one
2 · memory speed (bandwidth)	how fast a chat model can talk	this one
3 · compute (flops)	how fast it handles prompts and training	1, 5
4 · connection	how far one model can spread	1, 5

> Decision order: **fit** → **speed** → **compute** → **connect**. If it does not fit, nothing else matters.

3 GPU memory sizes



GPU memory: vendor specifications as of 14 Oct 2026, not measured here. Three tiers: 80 · about 141 to 192 · 288 GB.

4 Parameters and weights

A model is a giant list of numbers. Each number is a **parameter** (a weight). A 70B model has 70,000,000,000 of them. Training finds good values. **Inference** loads them all into memory and does arithmetic with them.

5 Weights in gigabytes

weights = parameters × bytes per parameter

Model	BF16 (2 B)	FP8 (1 B)	FP4 (½ B)
8B	16 GB	8 GB	4 GB
70B	**140 GB**	**70 GB**	35 GB
405B	810 GB	405 GB	202 GB

Arithmetic, decimal gigabytes. Script: l01_fit_math.py.

6 Precision: bytes per number

Each number is stored at a **precision**. Think of writing a price: £2.49 or rounded to £2. Rounding takes less space and loses a little detail.

Format	Bytes per number	Effect
BF16	2	the common default
FP8	1	half the memory of BF16
FP4	½	a quarter of BF16, on newer chips

Remember
Halve the bytes and you halve the memory. Newer chips can also **do the arithmetic** in the smaller formats.

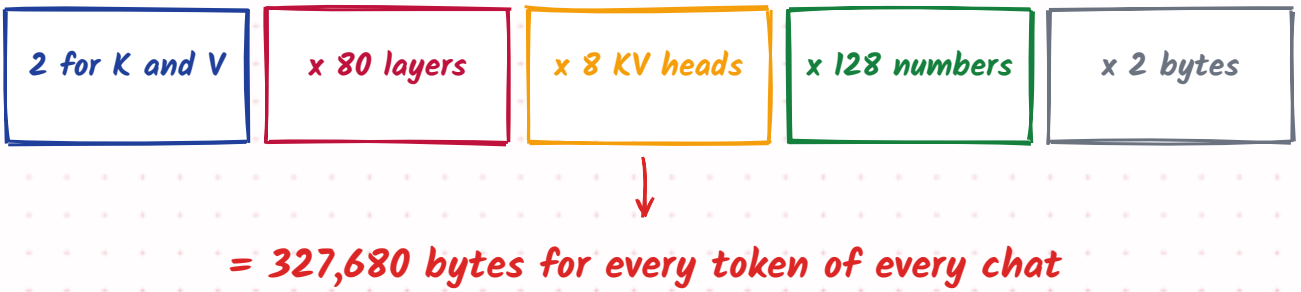
7 Does 70B fit?

- ✓ 70B in **FP8 = 70 GB**: fits an 80 GB H100, only just, with no room for anything else
- ✓ 70B in **BF16 = 140 GB**: does not fit an H100; borderline on an H200 (141 GB)
- ✓ Fixes: two cards, a bigger card, or a smaller format

> **Watch out**
> Weights fitting is **not enough**. The next page shows what the weights alone leave out.

8 The KV cache: a notepad per chat

> To write a reply the model keeps notes for the conversation so far: the **KV cache** (keys and values). Every token of every open chat adds to it, and it must stay in GPU memory while the chat is open.



> **KV bytes per token = 2 × layers × KV heads × head size × bytes per number**

Assumed 70B-class shape: 80 layers, 8 KV heads, head size 128. Typical, not any one model. Assumed 8B-class shape: 32 layers, 8 KV heads, 128.

9 Notes per chat

Config (assumed)	Bytes / token	8,192-token chat	32 chats
70B-class, BF16 notes	327,680	2.68 GB	85.9 GB
70B-class, FP8 notes	163,840	1.34 GB	**42.9 GB**
8B-class, BF16 notes	131,072	1.07 GB	34.4 GB
8B-class, FP8 notes	65,536	0.54 GB	17.2 GB

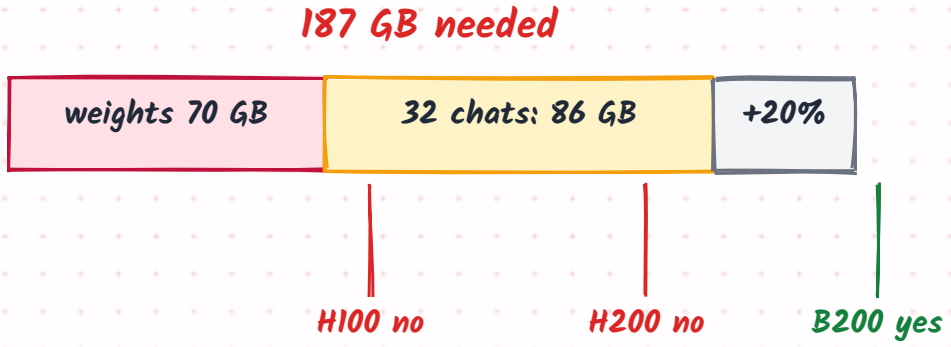
Arithmetic, l01_fit_math.py. A pattern, not a benchmark.

10 The fit rule

weights + KV cache for every open chat + 20 to 40% headroom (activations, graph captures, fragmentation).

Remember
The question is never just does the model fit. It is do the model and its chats fit.

11 Worked example: 70B, 32 chats



GPU	Memory	Needed 187 GB
H100	80 GB	no (234%)
H200	141 GB	no (133%)
B200	192 GB	**yes (97%)**
B300	288 GB	**yes (65%)**

70 GB FP8 weights + 85.9 GB BF16 notes (32 chats × 8,192 tokens) = 155.9 GB, × 1.2 headroom = 187 GB.

Ways to shrink the notes

→ Store notes in FP8: 85.9 GB becomes 42.9 GB (arithmetic above)

→ Share notes across heads (already counted: 8 KV heads)

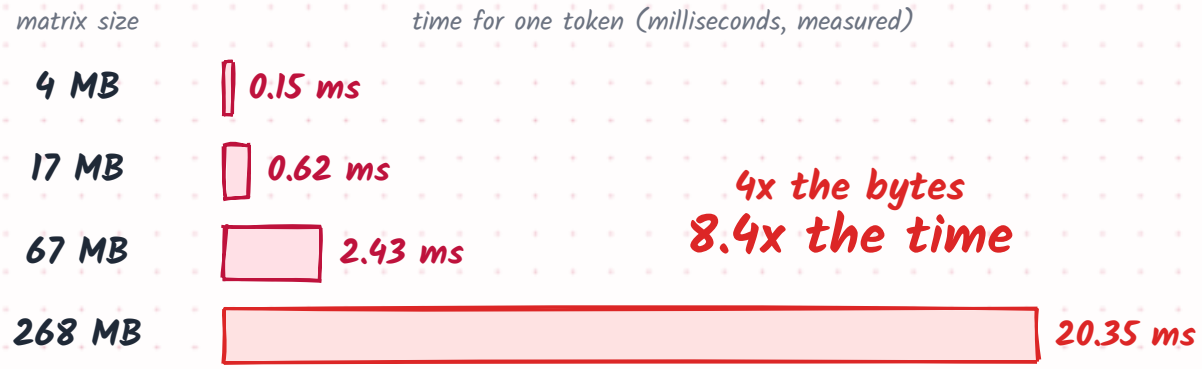
→ Hand notes out in small pages so no chat wastes space (taught, not measured here · lesson 6)

12 Number two: memory speed

> A chef who must walk to the fridge for every dish: the **walk** sets the pace, not the hands. To make **one** new token the GPU reads **all** the weights once. The speed of that read is **bandwidth**.

> ***time per token ≥ weight bytes ÷ memory bandwidth*** a floor, not a promise

Measured on my CPU: one token = matrix × vector

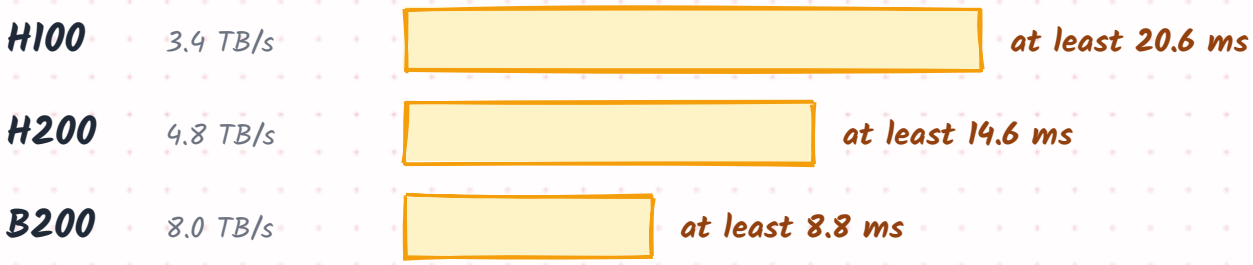


One core, float32, best of 7, whole run repeated: same story. A CPU, not a GPU. Script: l01_memory_bound.py.

The surprise
4x the bytes took **8.4x** the time (9.1x on the second run). The small tables sat in the chip's fast cache. The big one came from main memory, about half the speed. GPUs fix this with HBM next to the chip.

13 The same law on GPUs

70 GB of weights, read once per token



a floor from vendor specs, not a benchmark

GPU	Bandwidth	Floor per token	Max tokens/s
H100	3.4 TB/s	20.6 ms	49
H200	4.8 TB/s	14.6 ms	69
B200	8.0 TB/s	8.8 ms	114

70B in FP8 (70 GB), one chat at a time. Arithmetic from vendor bandwidth, not measured. Next generation is quoted near 22 TB/s, limited availability.

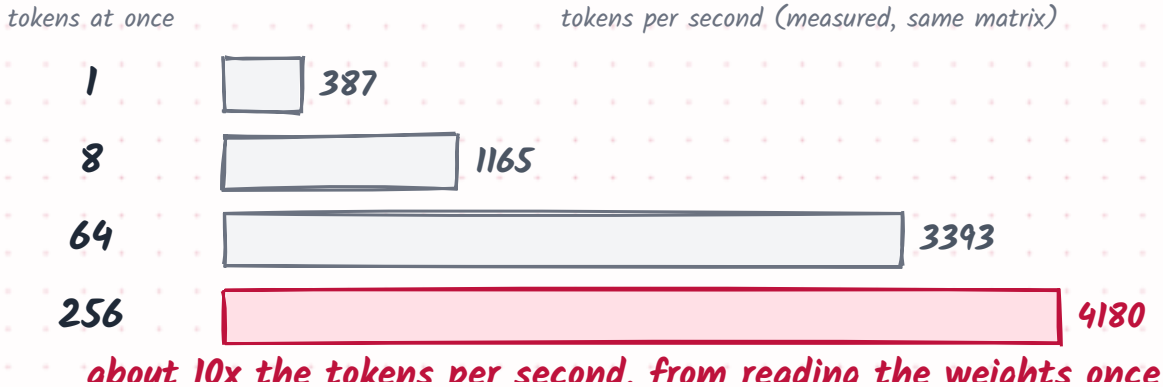
H200 vs H100
Same arithmetic power, more and faster memory. For chat that alone is a better buy.

14 Number three: compute

> **Compute** = how many multiplications and additions per second (flops). It matters when there is a lot of arithmetic per byte read.

Job	Arithmetic per byte	Limited by
reading a long prompt	high	compute
training	high	compute
writing a reply, one token at a time	low	memory speed

15 Batching: serve many at once

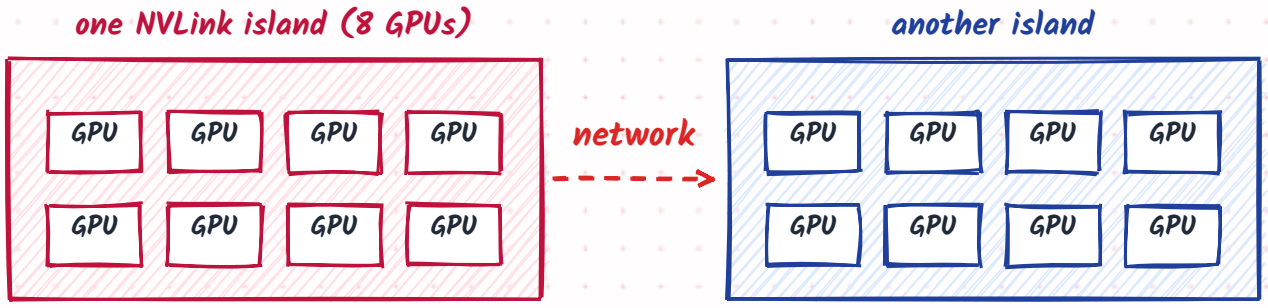


about 10x the tokens per second, from reading the weights once

Measured, 4096×4096 matrix. Second run: 388, 988, 2,860, 3,967 tokens/s. 256 at once = about 24x the time per step for 256x the tokens.

Linked limits
Every chat adds notes to memory. **Size** limits how many chats. **Speed** limits how fast each step runs.

16 Number four: connection



inside the island: about 450. between islands: about 50 (GB/s per GPU, H100)

→ Inside one server (8 GPUs on a vendor link): about 450 GB/s per GPU on H100

→ Between servers, over the network: about 50 GB/s per GPU, roughly **9x slower**

→ 405B in FP8 = 405 GB of weights = 8 × H100 (640 GB) in one server

Vendor figures, Oct 2026. Split a model across GPUs **inside** the fast island, not across the network.

17 Words to keep

Word	Meaning
Precision	how many bytes each number takes
KV cache	the notes kept for each open chat
Bandwidth	how fast memory can be read
HBM	high bandwidth memory, built next to the chip

18 ShopNest Copilot: the sum

Item	Size
weights, 8B in FP8	8 GB
32 chats × 8,192 tokens, FP8 notes (assumed 8B-class shape)	17.2 GB
20% headroom on top	total ≈ **30 GB**

That fits a single mid-size GPU. Lesson 2 shows which one to rent and what it costs.

19 Interview corner

Q. How do you size a GPU for a model?

A. Weights (parameters × bytes per parameter) plus the KV cache for every open chat plus 20 to 40% headroom.

Q. Why can a model fit the GPU and still fail in production?

A. The KV cache grows with every token of every open chat. 70B in FP8 is 70 GB, but 32 chats of 8,192 tokens add 86 GB of notes (BF16).

Q. Why does chat speed depend on memory speed?

A. Each new token reads all the weights once, so time per token is at least weight bytes ÷ bandwidth. Batching shares that read across users.

Q. When does the connection between GPUs matter?

A. When one model must be split across GPUs. Keep the split inside one fast server, not across the network, which is about 9x slower per GPU.

20 Interview questions

"Walk me through sizing GPU memory for a 70B chat model."

Weights first: 70B in FP8 is 70 GB. Add the KV cache for every open chat: 32 chats of 8,192 tokens add 85.9 GB in BF16. Add 20% headroom: about 187 GB, so one H100 or H200 is not enough.

Trap → Sizing on weights alone. The model fits, then crashes when the chats arrive.

"What is the KV cache, and how do you calculate its size?"

The keys and values kept for every token of every open chat, so earlier words are not recomputed. Bytes per token = 2 × layers × KV heads × head size × bytes per number: 327,680 bytes for the assumed 70B-class shape in BF16.

Trap → Forgetting the factor of 2 (keys AND values), or counting all attention heads when the model shares KV heads.

"How does precision change memory and speed?"

Bytes per number set both. BF16 is 2 bytes, FP8 1, FP4 half a byte. Halve the bytes and you halve the memory and the bytes read per token, so a memory-bound chat model also talks faster.

Trap → Assuming every chip does FP4 arithmetic: only newer chips do.

"Is a chat model limited by compute or by memory bandwidth?"

Writing a reply is memory-bound: each token reads all the weights once, so time per token ≥ weight bytes ÷ bandwidth. 70B FP8 on an H100 at 3.4 TB/s gives a floor of 20.6 ms, about 49 tokens per second for one chat.

Trap → Quoting peak flops for chat speed. Flops matter for long prompts and training.

"Why does batching make serving cheaper?"

All chats in a batch share one read of the weights. On my CPU test, 256 at once took about 24x the time per step for 256x the tokens.

Trap → Ignoring memory: every extra chat in the batch adds its own KV cache.

"How would you serve a 405B model?"

405B in FP8 is 405 GB of weights. Split it across 8 × H100 (640 GB) inside one server, where the link is about 450 GB/s per GPU. The network between servers is roughly 9x slower per GPU.

Trap → Splitting one model across servers when it fits inside one server.

21 Your turn

101_fit_math.py: change the model size, precision and number of chats and see which GPUs fit.

101_memory_bound.py: run on your own computer and compare your numbers with mine.

⚡ Revise in 60 seconds

- → 4 numbers: **size, speed, compute, connect**
- → weights = parameters × bytes (70B FP8 = **70 GB**)
- → KV cache: **327,680** bytes per token (assumed 70B-class)
- → 32 chats × 8,192 tokens = **85.9 GB** of notes (BF16)
- → time per token ≥ **weight bytes ÷ bandwidth**
- → CPU test: 4x the bytes took **8.4x** the time
- → batching: 1 → 256 at once = about **10x** tokens/s
- → inside the island ~450 GB/s · across it ~50 (H100)

Next lesson → CAI-02
Renting compute: which chip, what it costs, spot and autoscaling
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**